

AI radna stanica

Windows VM • VS Code • svi modeli • memorija

Kompletan vodič korak-po-korak: od praznog VM-a do okruženja u kome Claude, GPT, Gemini, OpenRouter i lokalni modeli rade zajedno, sa zajedničkim alatima (MCP) i trajnom memorijom.

Stanje alata: oktobar 2026 • proveriti verzije pre instalacije

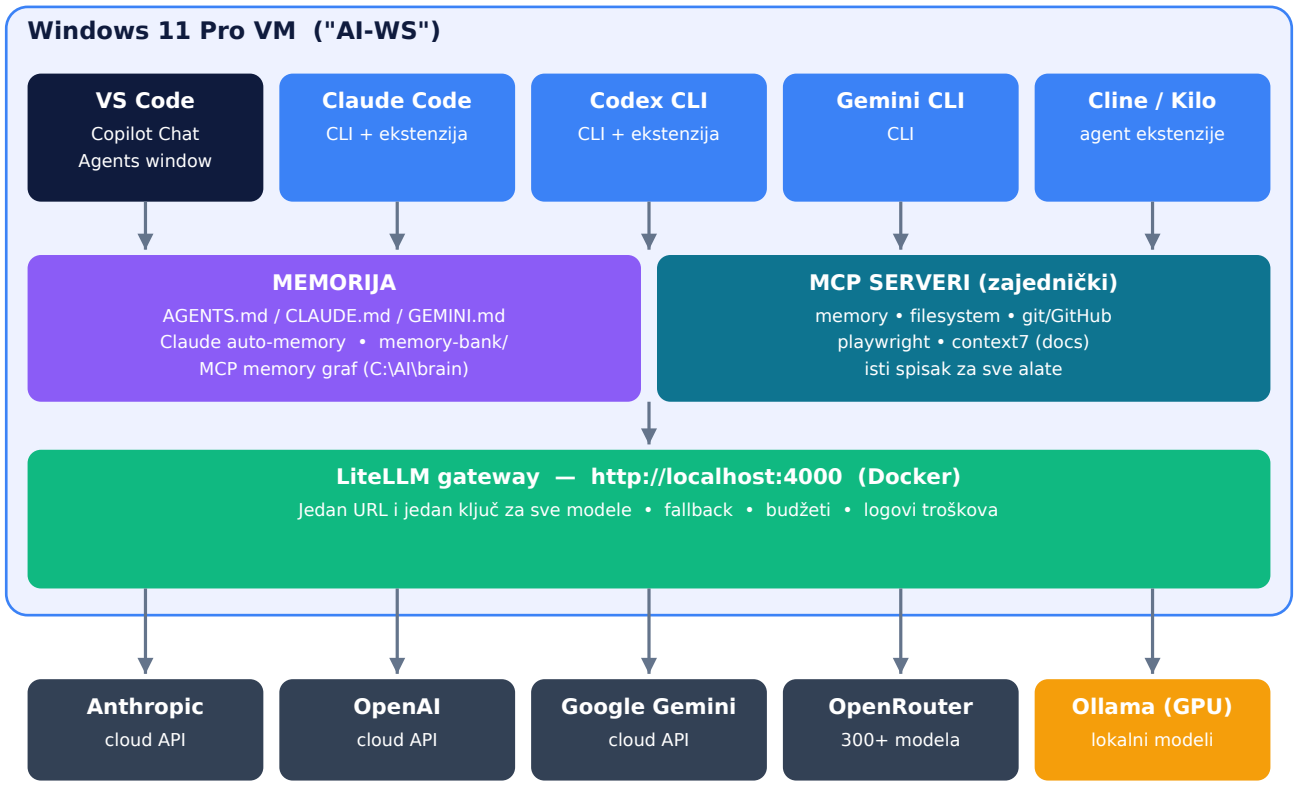
Jezik: srpski (latinica)

Sadržaj

1. Ukratko — preporučena postavka 3
2. Virtuelna mašina — specifikacija 4
3. Osnovni softver (jedna skripta) 5
4. API ključevi i bezbednost 6
5. Gateway: svi modeli preko jednog URL-a 6
6. Povezivanje alata sa modelima 8
7. Memorija — kako agenti „pamte“ 10
8. Zajednički MCP serveri 12
9. Svakodnevni rad: više agenata paralelno 13
10. Kontrolna lista 13
11. Izvori i napomene 14

1. Ukratko — preporučena postavka

Posle pregleda aktuelnih izvora (zvanična dokumentacija, release notes, uporedni testovi iz 2026.) najbolji praktičan pristup danas **nije jedan alat**, već **slojevita postavka**: nekoliko agenata u VS Code-u i terminalu, jedan zajednički *gateway* kroz koji idu svi modeli, zajednički MCP alati i memorija koja je ista za sve agente.



Sloj	Šta koristimo	Zašto
Agenti / UI	VS Code + GitHub Copilot (Agent mode, BYOK), Claude Code, Codex CLI, Gemini CLI, Cline ili Kilo Code	Svaki vendor ima najbolji agent za svoje modele; Copilot/Cline/Kilo pokrivaju sve ostale modele.
Pristup modelima	LiteLLM proxy (lokalno, Docker) + OpenRouter kao „sve ostalo“	Jedan OpenAI-kompatibilan URL za sve; fallback ako provajder padne; kontrola troškova.
Lokalni modeli	Ollama (Qwen3-Coder i sl.) na GPU-u	Autocomplete i privatni kod bez slanja u cloud, nula troška po tokenu.
Alati (MCP)	memory, filesystem, git/GitHub, Playwright, Context7	Isti alati dostupni svakom agentu — standard koji podržavaju svi.
Memorija	AGENTS.md kao jedan izvor istine + Claude auto-memory + MCP memory graf + memory-bank	Agent „pamti“ projekat, pravila i odluke između sesija i između alata.

Važno o stanju tržišta (2026)

• **Roo Code je ugašen** (repo arhiviran 15. maja 2026). Ne instalirati ga — naslednici su **Kilo Code** i **Cline** (oba aktivna).

• GitHub Copilot u VS Code-u od aprila 2026. ima **BYOK** (svoj API ključ: Anthropic, OpenAI, Gemini, OpenRouter, Ollama, Custom endpoint). Inline autocomplete i dalje ide preko GitHub-ovih modela.

• Za **Continue** postoje nepotvrđene vesti o akviziciji — koristiti ga samo kao opcioni dodatak.

2. Virtuelna mašina — specifikacija

Pošto su resursi dostupni, VM dimenzionišemo tako da istovremeno radi više agenata, Docker (gateway, baza) i indeksiranje koda. Lokalni modeli zahtevaju GPU — vidi napomenu ispod.

Resurs	Minimum	Preporuka	Napomena
OS	Windows 11 Pro	Windows 11 Pro / Enterprise 24H2+	Gen2 VM, Secure Boot, vTPM (obavezno za Win11)
vCPU	8	16–24	Paralelni agenti, Docker, build-ovi
RAM	32 GB	64–128 GB, **statički**	Dinamička memorija isključena (zbog nested virtualizacije)
Disk	256 GB SSD	512 GB – 1 TB NVMe	Modeli za Ollama zauzimaju 5–50 GB po modelu
GPU	—	NVIDIA 24 GB+ VRAM (passthrough)	Samo ako želite lokalne modele u samom VM-u
Mreža	NAT	Bridged + statička IP	Lakši pristup gateway-u sa drugih mašina

Hyper-V: kreiranje VM-a (PowerShell kao Administrator na hostu)

host • PowerShell (Admin)

```
$vm = "AI-WS"
New-VM -Name $vm -Generation 2 -MemoryStartupBytes 64GB `
       -NewVHDPath "D:\VMs\$vm\$vm.vhdx" -NewVHDSizeBytes 768GB -SwitchName "External"
Set-VMProcessor -VMName $vm -Count 16 -ExposeVirtualizationExtensions $true   # nested virt za
      Docker/WSL2
Set-VMMemory    -VMName $vm -DynamicMemoryEnabled $false
Set-VMKeyProtector -VMName $vm -NewLocalKeyProtector
Enable-VMTPM    -VMName $vm
Set-VMFirmware  -VMName $vm -EnableSecureBoot On
Add-VMDvdDrive  -VMName $vm -Path "D:\ISO\Win11_24H2.iso"
Start-VM $vm
```

GPU i lokalni modeli — realna preporuka
Na **Proxmox / ESXi** hostu: PCIe passthrough NVIDIA kartice direktno u VM — radi pouzdano.
Na **Hyper-V**: DDA postoji samo na Windows Server hostu; GPU-P na Windows 11 klijentu je nezvaničan.
Najčistija opcija: Ollama na zasebnoj Linux GPU mašini (ili na hostu), a VM se na nju kači preko `http://<gpu-host>:11434`. VM ostaje lagan, a GPU deli ceo tim.

Higijena VM-a

- Posle svake faze ovog vodiča napraviti **checkpoint/snapshot** (npr. „01-base“, „02-tools“, „03-ai-configured“).
- Poseban lokalni korisnik bez admin prava za svakodnevni rad; admin samo za instalacije.
- Uključiti BitLocker (vTPM) — na disku će biti API ključevi i kod.
- Kada je sve podešeno: exportovati VM kao **zlatnu sliku** i klonirati je za ostale članove tima.

3. Osnovni softver (jedna skripta)

Sve instaliramo preko **winget**-a da bi postavka bila ponovljiva. Pokrenuti u VM-u kao Administrator:

VM • PowerShell (Admin)

```
# 01-base.ps1  —  osnovni alati
$apps = @(
  "Microsoft.PowerShell", "Microsoft.WindowsTerminal", "Git.Git", "GitHub.cli",
  "Microsoft.VisualStudioCode", "OpenJS.NodeJS.LTS", "Python.Python.3.12", "astral-sh.uv",
  "Docker.DockerDesktop", "Ollama.Ollama", "BurntSushi.ripgrep.MSVC", "jqlang.jq",
  "Bitwarden.Bitwarden"   # ili drugi menadžer lozinki za API ključeve
)
foreach ($a in $apps) { winget install --id $a -e --accept-source-agreements --accept-package-
      agreements }

wsl --install -d Ubuntu          # WSL2 (Docker backend + Linux alati)
git config --global core.autocrlf input
git config --global init.defaultBranch main
# Developer Mode -> dozvoljava symlinkove bez admina (trebaće za deljenu memoriju)
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\AppModelUnlock" /t REG_DWORD /f /v
      "AllowDevelopmentWithoutDevLicense" /d "1"
Restart-Computer
```

CLI agenti

VM • PowerShell

```
# 02-agents.ps1  —  agenti u terminalu (novi PowerShell prozor posle restarta)
irm https://claude.ai/install.ps1 | iex      # Claude Code (native installer)
npm install -g @openai/codex                # OpenAI Codex CLI
npm install -g @google/gemini-cli           # Gemini CLI
claude --version; codex --version; gemini --version
```

Svaki CLI se prijavljuje pretplatom (Claude Pro/Max, ChatGPT Plus/Pro, Google nalog) ili API ključem. Za intenzivan agentski rad **pretplate su po pravilu dosta jeftinije od plaćanja po tokenu**; API ključeve čuvamo za gateway, automatizaciju i modele bez pretplate.

VS Code ekstenzije

VM • PowerShell

```
code --install-extension GitHub.copilot-chat
code --install-extension anthropic.claude-code
code --install-extension openai.chatgpt              # Codex ekstenzija
code --install-extension saoudrizwan.claude-dev      # Cline
code --install-extension kilocode.kilo-code          # Kilo Code (naslednik Roo Code-a)
code --install-extension ms-vscode-remote.remote-wsl
code --install-extension eamodio.gitlens
```

ID-jeve ekstenzija proveriti na Marketplace-u pre pokretanja — izdavači ponekad menjaju naziv. Ne instalirati sve agent-ekstenzije odjednom ako ne treba: **Cline ILI Kilo Code** je dovoljno.

4. API ključevi i bezbednost

- Ključeve držati u menadžeru lozinki; u VM ih upisati kao **korisničke environment varijable**, nikad u repo.
- Svaki provajder: postaviti **mesečni limit potrošnje** na svom dashboard-u (Anthropic, OpenAI, Google, OpenRouter).
- Za tim: svako dobija svoj **virtuelni ključ** na LiteLLM gateway-u (sa budžetom), a pravi ključevi ostaju samo na gateway-u.

VM • PowerShell

```
# 03-keys.ps1  –  jednom, kao običan korisnik
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY",  "sk-ant-...", "User")
[Environment]::SetEnvironmentVariable("OPENAI_API_KEY",     "sk-...",     "User")
[Environment]::SetEnvironmentVariable("GEMINI_API_KEY",     "AIza...",    "User")
[Environment]::SetEnvironmentVariable("OPENROUTER_API_KEY", "sk-or-...",  "User")
[Environment]::SetEnvironmentVariable("LITELLM_MASTER_KEY", "sk-local-" + [guid]::NewGuid(),
      "User")
```

Pravila za agente (stavite ih u AGENTS.md)
Agent nikad ne commit-uje `.env` i ključeve • destruktivne komande (brisanje, force push, drop baze) traže potvrdu • rad na posebnoj grani, PR umesto direktnog push-a na main. U Claude Code-u to se dodatno sprovodi kroz `permissions` u settings.json (deny pravila za opasne komande).

5. Gateway: svi modeli preko jednog URL-a

Ovo je deo koji omogućava da „jurite sve dostupne modele“. **LiteLLM** je open-source proxy koji izlaže jedan OpenAI-kompatibilan endpoint (`http://localhost:4000/v1`) i iza njega rutira ka bilo kom provajderu. Bilo koja ekstenzija koja podržava „OpenAI compatible“ provajdera (Copilot Custom Endpoint, Cline, Kilo, Continue…) odmah vidi sve modele. **OpenRouter** dodajemo kao „sve ostalo“ — stotine modela preko jednog ključa (uz malu proviziju), bez dodatnog podešavanja.

config.yaml

```
# C:\AI\gateway\config.yaml
model_list:
  # --- Anthropic ---
  - model_name: claude-opus
    litellm_params: { model: anthropic/claude-opus-5-5,   api_key: os.environ/ANTHROPIC_API_KEY }
  - model_name: claude-sonnet
    litellm_params: { model: anthropic/claude-sonnet-5-5, api_key: os.environ/ANTHROPIC_API_KEY }
  - model_name: claude-haiku
    litellm_params: { model: anthropic/claude-haiku-5-5,  api_key: os.environ/ANTHROPIC_API_KEY }
  # --- OpenAI / Google (upisati aktuelne ID-jeve modela sa njihovih stranica) ---
  - model_name: gpt
    litellm_params: { model: openai/<NAJNOVIJI-GPT>,         api_key: os.environ/OPENAI_API_KEY }
  - model_name: gemini-pro
    litellm_params: { model: gemini/<NAJNOVIJI-GEMINI-PRO>,  api_key: os.environ/GEMINI_API_KEY }
  # --- sve sa OpenRouter-a: npr. "openrouter/deepseek/..." ---
  - model_name: openrouter/*
    litellm_params: { model: openrouter/*, api_key: os.environ/OPENROUTER_API_KEY }
  # --- lokalno (Ollama na hostu ili GPU serveru) ---
  - model_name: local-coder
    litellm_params: { model: ollama_chat/qwen3-coder:30b, api_base: http://host.docker.internal:
      11434 }

router_settings:
  num_retries: 2
  fallbacks:
    - { claude-opus: [gpt, gemini-pro] }
    - { gpt: [claude-sonnet] }

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL        # potrošnja, virtuelni ključevi, budžeti
```

docker-compose.yml → cd C:\AI\gateway; docker compose up -d

```
# C:\AI\gateway\docker-compose.yml
services:
  litellm:
    image: ghcr.io/berriai/litellm:main-stable
    command: ["--config", "/app/config.yaml", "--port", "4000"]
    ports: ["4000:4000"]
    volumes: ["./config.yaml:/app/config.yaml:ro"]
    environment:
      - ANTHROPIC_API_KEY
      - OPENAI_API_KEY
      - GEMINI_API_KEY
      - OPENROUTER_API_KEY
      - LITELLM_MASTER_KEY
      - DATABASE_URL=postgresql://llm:llm@db:5432/litellm
    depends_on: [db]
    restart: unless-stopped
  db:
    image: postgres:16
    environment: { POSTGRES_USER: llm, POSTGRES_PASSWORD: llm, POSTGRES_DB: litellm }
    volumes: ["pgdata:/var/lib/postgresql/data"]
    restart: unless-stopped
volumes: { pgdata: {} }
```

Provera: `http://localhost:4000/ui` (admin panel: modeli, potrošnja, ključevi). Test iz PowerShell-a:

test

```
$h = @{ Authorization = "Bearer $env:LITELLM_MASTER_KEY" }
Invoke-RestMethod http://localhost:4000/v1/models -Headers $h | % data | % id
```

6. Povezivanje alata sa modelima

6.1 GitHub Copilot (VS Code) — BYOK

- Chat panel → birač modela → **Manage Models…** (otvara se Language Models editor).
- Dodati provajdere: **Anthropic**, **OpenAI**, **Gemini**, **OpenRouter**, **Ollama** — nalepiti ključ i označiti modele.
- Za gateway: provajder **Custom Endpoint / OpenAI Compatible**, URL `http://localhost:4000/v1`, ključ = LITELLM_MASTER_KEY (ili virtuelni ključ).
- U **Agent mode** se vide samo modeli koji podržavaju *tool calling*; lokalni model koji „nestane“ u Agent modu nema tu podršku.
- Ograničenja: inline autocomplete ostaje na GitHub modelima; admin organizacije može da isključi BYOK politikom.

6.2 Cline / Kilo Code

- Settings → API Provider: **OpenAI Compatible** → Base URL `http://localhost:4000/v1`, ključ, Model ID (npr. `claude-sonnet`).
- Alternativa bez gateway-a: provajder **OpenRouter** — direktno bira bilo koji od stotina modela.
- Kilo Code ima režime (Architect / Code / Debug / Ask) — za svaki režim može drugi model: skup model za planiranje, jeftin za izvršavanje.

6.3 Claude Code, Codex, Gemini CLI

- Najbolje rade sa **svojim** modelima i sopstvenom prijavom (pretplata) — tako ih i koristimo.
- Claude Code u VS Code-u: ekstenzija daje panel, diff pregled i deli kontekst otvorenih fajlova. Model se menja komandom `/model`.
- Opciono: Claude Code može kroz gateway (`ANTHROPIC_BASE_URL=http://localhost:4000`) radi centralnog logovanja troškova.
- Codex i Gemini CLI se mogu pokretati iz VS Code terminala; obe imaju i ekstenzije/integracije sa editorom.

6.4 Lokalni modeli (Ollama)

Ollama

```
ollama pull qwen3-coder:30b        # ~19 GB, MoE; komforno na 24 GB VRAM
ollama pull qwen3:14b              # za 12 GB kartice
ollama pull nomic-embed-text       # embeddings za indeksiranje koda / RAG
# Ollama na drugoj mašini: na toj mašini OLLAMA_HOST=0.0.0.0, pa u VM-u koristiti http://<gpu-
      host>:11434
```

Lokalni modeli su odlični za autocomplete, brza pitanja i poverljiv kod; za složene višekoračne agentske zadatke frontier cloud modeli su i dalje znatno pouzdaniji.

6.5 Koji model za šta

Zadatak	Preporuka	Alat
Veliki refaktor, nova funkcionalnost, agent koji radi satima	Claude Opus (ili Sonnet za brzinu/cenu)	Claude Code
Drugo mišljenje, code review, teški bagovi	Najnoviji GPT (Codex)	Codex CLI / Copilot
Ogroman kontekst (ceo monorepo, dugački logovi, PDF-ovi)	Gemini Pro	Gemini CLI / Copilot
Planiranje → izvršavanje	Skup model planira, jeftin izvršava	Kilo Code režimi / Cline Plan-Act
Isprobavanje novih modela (DeepSeek, Qwen, Mistral, Grok…)	OpenRouter	Cline / Kilo / Copilot BYOK
Autocomplete, privatni kod, offline	Qwen3-Coder lokalno	Ollama + Copilot/Continue/Kilo
Jeftini pomoćni poslovi (commit poruke, sažeci)	Claude Haiku ili mali lokalni model	bilo koji

Rangiranja modela se menjaju svakih par meseci — gateway sloj vam omogućava da promenite model u jednoj liniji config-a bez diranja alata.

7. Memorija — kako agenti „pamte“

Modeli sami ne pamte ništa između sesija. „Pamćenje“ se pravi od fajlova i alata koje agent čita na početku i dopunjuje tokom rada. Koristimo **četiri nivoa**:

Nivo	Šta je	Ko piše	Kada se učitava
1. Instrukcije	AGENTS.md (Codex, Copilot, Cline, Kilo...), CLAUDE.md (Claude), GEMINI.md (Gemini)	Vi (i agent na zahtev)	Automatski na početku svake sesije
2. Auto-memorija	Claude Code sam beleži naučeno u MEMORY.md pod ~/.claude/projects/...	Agent	Automatski (početak fajla)
3. Memory bank	Folder memory-bank/ u repou: kontekst, odluke, napredak (Markdown)	Agent po pravilu iz AGENTS.md	Agent ga čita na početku zadatka
4. MCP memory graf	Entiteti/relacije/opažanja u JSON-u, dostupni svim agentima kao alat	Agent preko MCP alata	Na zahtev (pretraga)

7.1 Jedan izvor istine za sve alate

Pravimo folder `C:\AI\brain` (git repo, može na privatni GitHub radi backup-a i deljenja između VM-ova). Globalna pravila pišemo jednom u `AGENTS.md`, a svaki alat dobija link ka tom fajlu:

VM • PowerShell (Developer Mode uključen)

```
# 04-memory.ps1
$brain = "C:\AI\brain"; New-Item -ItemType Directory -Force $brain, "$brain\memory" | Out-Null
cd $brain; git init

# globalna pravila -> svaki alat ih čita iz svog standardnog mesta
New-Item -ItemType Directory -Force "$HOME\.claude", "$HOME\.codex", "$HOME\.gemini" | Out-Null
New-Item -ItemType SymbolicLink -Force -Path "$HOME\.codex\AGENTS.md"  -Target "$brain\AGENTS.md"
New-Item -ItemType SymbolicLink -Force -Path "$HOME\.gemini\GEMINI.md" -Target "$brain\AGENTS.md"
Set-Content "$HOME\.claude\CLAUDE.md" "@$brain\AGENTS.md"     # Claude: import sintaksa @putanja
```

U svakom projektu isto: `AGENTS.md` u korenu repoa, a `CLAUDE.md` sadrži samo liniju `@AGENTS.md`. Za Gemini CLI u `~/.gemini/settings.json` dodati da čita i AGENTS.md (`"context": {"fileName": ["AGENTS.md", "GEMINI.md"]}`; u starijim verzijama ključ `contextFileName`).

AGENTS.md

```
# AGENTS.md  (globalni primer — kratko i konkretno, ispod ~200 linija)
## Ko sam i kako radim
- Odgovaraj na srpskom, kod i komentari na engleskom.
- Stack: <upišite: .NET / Node / Python...>. Testovi pre svakog commit-a.
## Pravila
- Nikad ne commit-uj tajne (.env, ključevi). Destruktivne komande samo uz potvrdu.
- Radi na posebnoj grani; male, opisne commit poruke.
## Memorija
- Na početku zadatka pročitaj memory-bank/ (ako postoji) i pretraži MCP "memory".
- Kada doneseš važnu odluku ili naučiš nešto o projektu: zapiši u memory-bank/
  (activeContext.md, progress.md) i dodaj opažanje u MCP memory graf.
```

7.2 Memory bank (po projektu)

Popularan obrazac iz Cline zajednice, radi sa svim agentima jer je samo Markdown u repou:

struktura

```
memory-bank/
  projectbrief.md     # šta pravimo i zašto (piše čovek)
  productContext.md   # korisnici, problemi, ciljevi
  systemPatterns.md   # arhitektura, ključne odluke (ADR ukratko)
  techContext.md      # stack, komande, okruženje
  activeContext.md    # na čemu se trenutno radi, sledeći koraci
  progress.md         # šta je gotovo, šta ne radi, poznati bagovi
```

7.3 MCP memory graf (deljen između svih agenata)

Zvanični `@modelcontextprotocol/server-memory` čuva entitete i relacije u jednom JSON fajlu. Pošto svi alati gađaju **isti fajl**, ono što Claude zapamti, vidi i Copilot/Codex/Gemini.

8. Zajednički MCP serveri

MCP (Model Context Protocol) je standard kojim agenti dobijaju alate. Podesimo isti skup u svakom alatu:

Server	Paket / URL	Čemu služi
memory	@modelcontextprotocol/server-memory	Trajna memorija (graf znanja)
filesystem	@modelcontextprotocol/server-filesystem	Pristup dozvoljenim folderima (npr. C:\AI, C:\src)
github	https://api.githubcopilot.com/mcp/	Issues, PR-ovi, review (zvanični GitHub MCP, remote)
playwright	@playwright/mcp	Agent otvara browser, testira UI, pravi screenshot-ove
context7	@upstash/context7-mcp	Aktuelna dokumentacija biblioteka (manje izmišljenih API-ja)

mcp.json (VS Code)

```
// VS Code (Copilot, i ekstenzije koje čitaju VS Code MCP):  %APPDATA%\Code\User\mcp.json
// ili po projektu: .vscode/mcp.json
{
  "servers": {
    "memory": {
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-memory"],
      "env": { "MEMORY_FILE_PATH": "C:\\AI\\brain\\memory\\memory.json" }
    },
    "filesystem": {
      "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem", "C:\\AI", "C:\\
      src"]
    },
    "playwright": { "command": "npx", "args": ["-y", "@playwright/mcp@latest"] },
    "context7":   { "command": "npx", "args": ["-y", "@upstash/context7-mcp"] },
    "github":     { "type": "http", "url": "https://api.githubcopilot.com/mcp/" }
  }
}
```

Claude Code

```
# Claude Code — korisnički nivo (važi u svim projektima). Na Windows-u npx ide kroz "cmd /c".
claude mcp add memory -s user -e MEMORY_FILE_PATH=C:\AI\brain\memory\memory.json -- cmd /c npx -y
      @modelcontextprotocol/server-memory
claude mcp add playwright -s user -- cmd /c npx -y @playwright/mcp@latest
claude mcp add context7   -s user -- cmd /c npx -y @upstash/context7-mcp
claude mcp list
```

Codex + Gemini

```
# Codex — %USERPROFILE%\.codex\config.toml
[mcp_servers.memory]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-memory"]
env = { MEMORY_FILE_PATH = 'C:\AI\brain\memory\memory.json' }

// Gemini CLI — %USERPROFILE%\.gemini\settings.json
{ "context": { "fileName": ["AGENTS.md", "GEMINI.md"] },
  "mcpServers": { "memory": { "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"],
      "env": { "MEMORY_FILE_PATH": "C:\\AI\\brain\\memory\\memory.json" } } } }
```

Cline i Kilo Code imaju svoj MCP panel (ikonica servera u ekstenziji → Edit config) sa istim `mcpServers` formatom kao gore. Ne dodavati desetine servera: svaki alat troši kontekst modela — 5–8 dobro izabranih je optimum.

9. Svakodnevni rad: više agenata paralelno

- **Git worktree po agentu**: svaki agent radi u svom folderu i grani, bez sudaranja — `git worktree add ..\proj-feat-x -b feat/x`.
- **VS Code Agents window** i Claude Code sub-agenti: jedan zadatak se deli na delove (istraživanje, implementacija, testovi).
- **Unakrsni review**: kod koji napiše Claude da pregleda GPT (Codex), i obrnuto — hvata greške koje jedan model sistematski ne vidi.
- **Plan pre koda**: za veće zadatke tražiti plan (Claude Code plan mode, Kilo Architect, Cline Plan), pa tek onda izvršavanje.
- **Kraj sesije**: „ažuriraj memory-bank i zapamti odluke“ — tako sledeća sesija (bilo kog agenta) kreće od iste tačke.
- **Troškovi**: Agent mod troši višestruko više tokena od običnog chata; pratiti dashboard na `localhost:4000/ui` i provajderske limite.

Struktura foldera u VM-u

```layout
C:\AI\
  brain\                # git repo: globalni AGENTS.md, memory\memory.json, beleške
  gateway\              # LiteLLM config.yaml + docker-compose.yml
  scripts\              # 01-base.ps1, 02-agents.ps1, 03-keys.ps1, 04-memory.ps1
C:\src\                 # projekti (svaki sa AGENTS.md, CLAUDE.md -> @AGENTS.md, memory-bank\)
```

10. Kontrolna lista

- [ ] VM: Gen2, vTPM, 16+ vCPU, 64 GB statički RAM, nested virtualization, BitLocker
- [ ] winget osnovni paket + WSL2 + Docker Desktop radi (`docker run hello-world`)
- [ ] Claude Code, Codex CLI, Gemini CLI instalirani i prijavljeni
- [ ] VS Code: Copilot Chat, Claude Code, Codex, Cline ili Kilo Code
- [ ] API ključevi kao User env varijable + limiti potrošnje kod provajdera
- [ ] LiteLLM gateway gore, `/v1/models` vraća sve modele, fallback testiran
- [ ] Copilot BYOK + Cline/Kilo povezani na gateway ili OpenRouter
- [ ] Ollama: lokalni coder model + embeddings (u VM-u ili na GPU serveru)
- [ ] C:\AI\brain: globalni AGENTS.md povezan za Claude/Codex/Gemini
- [ ] MCP serveri (memory, filesystem, github, playwright, context7) u svim alatima, isti memory.json
- [ ] Test: kažete Claude-u „zapamti X“, pa pitate Codex/Copilot „šta znaš o X?“ — mora da zna
- [ ] Snapshot „03-ai-configured“ + export zlatne slike

11. Izvori i napomene

Vodič je zasnovan na pretrazi aktuelnih izvora (oktobar 2026). Deo izvora su blogovi i uporedni sajtovi; ključne tvrdnje su proverene kroz više izvora, ali **pre instalacije proverite zvaničnu dokumentaciju** — alati se menjaju mesečno.

GitHub Copilot BYOK u VS Code-u (Anthropic, Ollama, Foundry Local)
https://startdebugging.net/2026/04/github-copilot-vs-code-byok-anthropic-ollama-foundry-local/

Copilot Agent mode — vodič
https://lucaberton.com/blog/github-copilot-agent-mode-vscode-complete-guide/

GitHub release notes (BYOK, Custom Endpoint, Agents window)
https://releasebot.io/updates/github

Kilo Code vs Roo Code posle gašenja
https://kilo.ai/kilo-code/vs/roo-code

Cline vs Kilo Code vs Roo Code (2026)
https://agentscamp.com/guides/comparisons/cline-vs-kilo-code-vs-roo-code

Roo Code vs Cline — razlike i migracija
https://fast.io/resources/roo-code-vs-cline/

Open-source AI coding asistenti — vodič 2026
https://www.morphllm.com/ai-coding-assistant-open-source

OpenRouter vs LiteLLM (2026)
https://gateway.orq.ai/blog/openrouter-vs-litellm

Portkey vs LiteLLM vs OpenRouter
https://www.pkgpulse.com/guides/portkey-vs-litellm-vs-openrouter-llm-gateway-2026

Agentic primitives: Codex CLI vs Claude Code vs Gemini CLI
https://codex.danielvaughan.com/2026/03/26/agentic-primitives-codex-claude-gemini/

Claude Code vs Codex CLI vs Gemini CLI
https://www.deployhq.com/blog/comparing-claude-code-openai-codex-and-google-gemini-cli-which-ai-coding-assistant-is-right-for-your-deployment-workflow

Knowledge Graph Memory MCP server
https://playbooks.com/mcp/modelcontextprotocol-knowledge-graph-memory

Docker MCP — memory server
https://hub.docker.com/mcp/server/memory/manual

LLM-ovi na Windows-u — podešavanje
https://llmhardware.io/guides/how-to-run-llms-on-windows

Ollama — coding modeli
https://registry.ollama.ai/blog/coding-models

Zvanično: Claude Code docs
https://code.claude.com/docs

Zvanično: LiteLLM proxy docs
https://docs.litellm.ai/docs/simple_proxy

Zvanično: MCP serveri
https://github.com/modelcontextprotocol/servers

Nepotvrđeno / pratiti

• Vesti o akviziciji Continue-a (jun 2026) — nisu potvrđene iz zvaničnih izvora.
• Navodi o premeštanju besplatnog Gemini CLI nivoa u zaseban alat — proveriti na Google stranicama.
• ID-jeve GPT i Gemini modela u config.yaml upisati sa zvaničnih stranica provajdera u trenutku instalacije.